

SkillSwap

Complete Project Documentation

Student-to-Student Skill Marketplace

Version: 1.0.0

Path: D:\Projects\SkillSwap

Backend: Django 6.1 + DRF 3.18 + PostgreSQL

Frontend: React 19 + Vite 8 + TypeScript + Tailwind 4

Generated: September 2026

Table of Contents

1.	Project Overview	3
2.	Tech Stack	3
3.	Project Structure & File Map	4
4.	How to Run the Project	6
5.	Environment Variables (.env)	8
6.	Database Schema (All Models)	10
7.	Authentication System (JWT + OTP)	15
8.	API Routes - Complete Reference	17
9.	Frontend Architecture	22
10.	Pages & Routing (21 Pages)	23
11.	Components Reference	26
12.	State Management	28
13.	Business Logic & State Machines	29
14.	Payments (Stripe + Simulation)	32
15.	Messaging System	33
16.	Notifications System	34
17.	Moderation & Reports	35
18.	How to Edit/Modify Everything	36
19.	Deployment (Docker)	42
20.	Testing	43
21.	Troubleshooting	44

1. Project Overview

SkillSwap is a student-to-student skill marketplace platform. It allows university students to offer and purchase skills/services from each other. Think of it as a campus-focused Fiverr/Upwork.

Core Features

- User registration with email verification (OTP codes)
- Student profiles with skills, bio, university info, avatar
- Service listings with categories, pricing, delivery time
- Application system (students apply to buy a service)
- Order management with payment (Stripe or simulation mode)
- Messaging threads per order (real-time chat)
- Star rating reviews on completed orders
- In-app notifications for all lifecycle events
- Content moderation (reports, staff suspension, listing review)
- Admin dashboard for staff moderation
- JWT authentication with token rotation and blacklisting
- Full REST API with OpenAPI/Swagger documentation

User Roles

- Guest: Browse marketplace, view public profiles/listings
- Student (email-verified): Create listings, apply, order, pay, message, review
- Staff/Admin: Moderate listings, resolve reports, suspend users

2. Tech Stack

Backend

Component	Technology	Version
Framework	Django	6.1.1
API	Django REST Framework	3.18.0
Database	PostgreSQL (psycopg3)	16 / 3.3.5
Auth	djangorestframework-simplejwt	5.5.1
API Docs	drf-spectacular (OpenAPI 3)	0.28.0
Payments	Stripe SDK	15.6.1
Images	Pillow	12.3.0
CORS	django-cors-headers	4.9.0
Env Vars	python-dotenv	1.2.3
Server (prod)	gunicorn	23.0.0
Linting	ruff	0.16.6
Testing	pytest + pytest-django	9.1.1
Python	Python	3.13

Frontend

Component	Technology	Version
Framework	React	19.x
Build Tool	Vite	8.x
Language	TypeScript	6.0
Routing	react-router-dom	7.x
Server State	@tanstack/react-query	5.x
Forms	react-hook-form + zod	7.x / 3.x
HTTP Client	axios	1.x
Styling	Tailwind CSS	4.x
Testing	Vitest + Testing Library	latest
Linting	oxlint	latest

3. Project Structure & File Map

Top-Level Directory

```
D:\Projects\SkillSwap\
.env                # Environment variables (secrets)
.env.example        # Template for .env
docker-compose.yml  # Docker orchestration
README.md           # Project readme
docs/ARCHITECTURE.md # Architecture docs
.github/workflows/   # CI pipeline
backend/            # Django backend
frontend/           # React frontend
```

Backend Structure

```
backend/
manage.py           # Django CLI entry point
pyproject.toml      # pytest + ruff config
Dockerfile          # Production container
requirements/
  base.txt           # Runtime dependencies
  dev.txt            # Dev dependencies (pytest, ruff)
  production.txt     # Production (gunicorn)
config/
  settings.py        # ALL settings (single file)
  urls.py            # Root URL router
  wsgi.py / asgi.py  # Server entry points
apps/
  core/              # Health check, validators
  users/             # User model, JWT auth, OTP
  profiles/          # Student profiles, skills
  listings/          # Service listings, applications
  orders/            # Orders, payments, Stripe
  messaging/         # Chat threads & messages
  notifications/     # In-app notifications
  reviews/           # Star ratings
  moderation/        # Reports, staff actions
```

Each Django App Contains

```
apps/<app>/
models.py           # Database models (tables)
views.py            # API endpoint handlers
serializers.py       # Data validation & serialization
services.py         # Business logic (the real logic)
urls.py             # URL patterns for this app
admin.py            # Django admin panel config
permissions.py      # Custom permission classes
tests.py or test_api.py # Tests
```

Frontend Structure

```
frontend/
  package.json      # Dependencies & scripts
  vite.config.ts    # Vite build config
  tsconfig.json     # TypeScript config
  Dockerfile        # Multi-stage build (node->nginx)
  nginx.conf        # Production web server config
src/
  main.tsx          # App entry point & providers
  App.tsx           # All route definitions
  index.css         # Tailwind + custom theme
  types/index.ts    # TypeScript type definitions
api/
  client.ts         # Axios instance + interceptors
services/           # API service modules
  auth.ts           # Login, register, OTP, password
  profiles.ts       # Profile CRUD, skills
  listings.ts       # Listings, applications
  orders.ts         # Orders, checkout
  messaging.ts      # Threads, messages
  notifications.ts  # Notifications
  reviews.ts       # Reviews
  reports.ts        # Reports + admin moderation
hooks/
  useAuth.tsx       # Auth context provider
routes/
  guards.tsx        # ProtectedRoute, AdminRoute
pages/              # 21 page components
components/
  common/           # Button, Field, Avatar, etc.
  layout/           # Navbar, Layout
  listings/         # ListingCard
lib/
  queryClient.ts    # React Query config
utils/
  format.ts         # Formatting helpers
```

4. How to Run the Project

Option A: Docker (Recommended)

This runs everything (database, backend, frontend) in containers:

```
# 1. Copy and configure environment variables
copy .env.example .env
# Edit .env with your settings

# 2. Start everything
docker compose up --build

# 3. Access:
# Frontend: http://localhost:8080
# Backend: http://localhost:8000
# API Docs: http://localhost:8000/api/v1/schema/swagger-ui/
# Admin: http://localhost:8000/admin/
```

Option B: Manual (Development)

Prerequisites

- Python 3.13+
- Node.js 22+
- PostgreSQL 16+
- Redis (optional, for caching)

Backend Setup

```
# 1. Navigate to backend
cd backend

# 2. Create virtual environment
python -m venv .env
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Mac/Linux

# 3. Install dependencies
pip install -r requirements/dev.txt

# 4. Configure .env (in project root)
# Set DATABASE_NAME, DATABASE_USER, DATABASE_PASSWORD
# Set DJANGO_SECRET_KEY

# 5. Create database
psql -U postgres -c "CREATE DATABASE skillswap_db;"

# 6. Run migrations
python manage.py migrate

# 7. Create admin user
python manage.py createsuperuser

# 8. Start backend server
python manage.py runserver
# Runs at http://localhost:8000
```

Frontend Setup

```
# 1. Navigate to frontend
cd frontend

# 2. Install dependencies
npm install

# 3. Start dev server
npm run dev
# Runs at http://localhost:5173
```

Available Scripts

Backend (from backend/)

python manage.py runserver	# Dev server
python manage.py migrate	# Apply migrations
python manage.py makemigrations	# Create new migrations
python manage.py createsuperuser	# Create admin
python manage.py spectacular --validate	# Validate API schema
pytest	# Run tests
ruff check .	# Lint code
ruff format .	# Format code

Frontend (from frontend/)

```
npm run dev      # Dev server (Vite)
npm run build    # Production build
npm run preview  # Preview production build
npm run lint     # Run oxlint
npx vitest      # Run tests
```

5. Environment Variables (.env)

All configuration is in a single .env file at the project root. Copy .env.example to .env and edit.

Variable	Default	Description
DJANGO_SECRET_KEY	(dev fallback)	Django secret (REQUIRED in prod)
DEBUG	True	Debug mode
DJANGO_ALLOWED_HOSTS	localhost,127.0.0.1	Comma-separated hosts
DATABASE_NAME	skillswap_db	PostgreSQL database name
DATABASE_USER	postgres	PostgreSQL username
DATABASE_PASSWORD	change-me	PostgreSQL password
DATABASE_HOST	localhost	PostgreSQL host
DATABASE_PORT	5432	PostgreSQL port
JWT_SECRET_KEY	(uses SECRET_KEY)	JWT signing key
JWT_ACCESS_TOKEN_MINUTES	30	Access token lifetime
JWT_REFRESH_TOKEN_DAYS	30	Refresh token lifetime
CORS_ALLOWED_ORIGINS	http://localhost:5173	CORS origins
EMAIL_BACKEND	console.EmailBackend	Email backend
EMAIL_HOST	(empty)	SMTP host
EMAIL_PORT	587	SMTP port
EMAIL_HOST_USER	(empty)	SMTP username
EMAIL_HOST_PASSWORD	(empty)	SMTP password
DEFAULT_FROM_EMAIL	SkillSwap <no-reply@...>	Sender email
OTP_CODE_VALIDITY_SECONDS	600	OTP validity (10 min)
STRIPE_API_KEY	(empty)	Stripe key (empty=simulation)
STRIPE_WEBHOOK_SECRET	(empty)	Stripe webhook secret
STRIPE_SUCCESS_URL	http://localhost:5173/...	Stripe redirect
STRIPE_CANCEL_URL	http://localhost:5173/...	Stripe redirect
VITE_API_URL	/api/v1	Frontend API base URL
REDIS_URL	redis://localhost:6379/0	Redis (optional)

Important Notes

- When STRIPE_API_KEY is empty, the app uses SimulationGateway (mock payments)
- When EMAIL_BACKEND is 'console', emails print to terminal (no real email)
- DEBUG=True enables media file serving and allows all hosts
- VITE_API_URL is read at build time for the frontend

6. Database Schema (All Models)

The database has 12 models across 9 Django apps. All use PostgreSQL. Migrations are in each app's migrations/ folder.

6.1 User (apps.users)

Extends Django's AbstractUser. Custom user model.

Field	Type	Notes
id	BigAutoField	Primary key (auto)
username	CharField(150)	Unique, inherited
email	EmailField	Unique, used for login
password	CharField(128)	Hashed, inherited
first_name	CharField(150)	Inherited
last_name	CharField(150)	Inherited
email_verified	BooleanField	Default: False
is_active	BooleanField	False = suspended
is_staff	BooleanField	Admin panel access
date_joined	DateTimeField	Auto-set on creation

6.2 OneTimePassword (apps.users)

Stores hashed OTP codes for email verification and password reset.

Field	Type	Notes
id	BigAutoField	Primary key
user	FK -> User	CASCADE
purpose	CharField(32)	email_verification password_reset
hashed_code	CharField(128)	make_password() hash
expires_at	DateTimeField	Expiry timestamp
attempts	PositiveIntegerField	Default: 0, max 5
is_used	BooleanField	Default: False
created_at	DateTimeField	Auto-set

Constraint: Only one active (is_used=False) OTP per user per purpose.

6.3 Skill (apps.profiles)

Global skill tags that users and listings can reference.

Field	Type	Notes
id	BigAutoField	Primary key
name	CharField(80)	Unique
slug	SlugField(100)	Auto-generated from name
created_at	DateTimeField	Auto-set

6.4 Profile (apps.profiles)

One-to-one with User. Contains student-specific data.

Field	Type	Notes
id	BigAutoField	Primary key
user	OneToOne -> User	CASCADE, related_name='profile'
avatar	ImageField	upload_to='avatars/', optional
university	CharField(150)	Blank OK
department	CharField(150)	Blank OK
is_student	BooleanField	Default: True
bio	TextField(500)	Blank OK
location	CharField(120)	Blank OK
experience_years	SmallInt	Default: 0
skills	M2M -> Skill	related_name='profiles'
is_searchable	BooleanField	Default: True (privacy)
is_verified_student	BooleanField	Staff-only badge
created_at	DateTimeField	Auto-set
updated_at	DateTimeField	Auto-updated

Property: display_name returns 'First Last' or username.

6.5 Category (apps.listings)

Field	Type	Notes
id	BigAutoField	Primary key
name	CharField(80)	Unique
slug	SlugField(100)	Auto-generated
description	CharField(300)	Blank OK
is_active	BooleanField	Default: True
created_at	DateTimeField	Auto-set

6.6 Listing (apps.listings)

A service/skill listing offered by a student.

Field	Type	Notes
id	BigAutoField	Primary key
provider	FK -> Profile	CASCADE (the seller)
category	FK -> Category	SET_NULL, nullable
title	CharField(120)	Required
slug	SlugField(130)	Unique, auto from title
description	TextField(2000)	Blank OK
price	Decimal(8,2)	Required
currency	CharField(3)	Default: 'USD'
delivery_time_days	SmallInt	Nullable
is_remote	BooleanField	Default: True
location	CharField(120)	Blank OK
skills	M2M -> Skill	Tags for the listing
cover_image	ImageField	upload_to='listings/'
is_active	BooleanField	Default: True
is_archived	BooleanField	Soft delete flag
moderation_status	CharField	draft/pending/published/rejected/suspended/archived
created_at	DateTimeField	Auto-set
updated_at	DateTimeField	Auto-updated

Method: archive() sets is_active=False, is_archived=True (soft delete). Slug auto-handles collisions with -2, -3 suffixes.

6.7 Application (apps.listings)

A student's request to buy a listing service.

Field	Type	Notes
id	BigAutoField	Primary key
listing	FK -> Listing	CASCADE
applicant	FK -> Profile	CASCADE (the buyer)
message	TextField(500)	Optional message to seller
proposed_price	Decimal(8,2)	Counter-offer, nullable
status	CharField(12)	pending/accepted/rejected/withdrawn
responded_at	DateTimeField	Nullable
created_at	DateTimeField	Auto-set
updated_at	DateTimeField	Auto-updated

Constraint: Only one pending application per applicant per listing.

6.8 Order (apps.orders)

Created when an application is accepted. Goes through a state machine.

Field	Type	Notes
id	BigAutoField	Primary key
application	OneToOne -> Application	PROTECT
buyer	FK -> Profile	CASCADE
provider	FK -> Profile	CASCADE
listing	FK -> Listing	SET_NULL, nullable
price	Decimal(8,2)	Snapshot of agreed price
currency	CharField(3)	Default: 'USD'
note	TextField(500)	Buyer's note, optional
status	CharField(20)	pending_payment/paid/in_progress/...
paid_at	DateTimeField	Nullable
started_at	DateTimeField	Nullable
completed_at	DateTimeField	Nullable
cancelled_at	DateTimeField	Nullable

6.9 Payment (apps.orders)

Field	Type	Notes
id	BigAutoField	Primary key
order	OneToOne -> Order	PROTECT
amount	Decimal(8,2)	Payment amount
currency	CharField(3)	Default: 'USD'
status	CharField(10)	created/paid/refunded/failed
gateway	CharField(20)	Default: 'stripe'
gateway_session_id	CharField(255)	Stripe session ID
gateway_charge_id	CharField(255)	Stripe charge ID

6.10 Thread (apps.messaging)

One conversation per order, between buyer and provider.

Field	Type	Notes
id	BigAutoField	Primary key
order	OneToOne -> Order	CASCADE
buyer	FK -> Profile	CASCADE
provider	FK -> Profile	CASCADE
last_message_at	DateTimeField	Nullable, for sorting

6.11 Message (apps.messaging)

Field	Type	Notes
id	BigAutoField	Primary key
thread	FK -> Thread	CASCADE
sender	FK -> Profile	CASCADE
body	TextField(2000)	Message text
is_read	BooleanField	Default: False
read_at	DateTimeField	Nullable

6.12 Notification (apps.notifications)

Generic in-app notifications for all lifecycle events.

Field	Type	Notes
id	BigAutoField	Primary key
recipient	FK -> User	CASCADE
actor	FK -> User	CASCADE, nullable
verb	CharField(64)	e.g. new_order, order_paid
target_type	CharField(50)	order, application, thread...
target_id	BigAutoField	ID of the target object
data	JSONField	Arbitrary payload
is_read	BooleanField	Default: False
read_at	DateTimeField	Nullable

6.13 Review (apps.reviews)

One review per completed order, by buyer on provider.

Field	Type	Notes
id	BigAutoField	Primary key
order	OneToOne -> Order	CASCADE
reviewer	FK -> Profile	CASCADE (buyer)
reviewee	FK -> Profile	CASCADE (provider)
rating	SmallInt	1-5, validated
comment	TextField(1000)	Optional text

6.14 Report (apps.moderation)

Polymorphic reports against users, listings, messages, or reviews.

Field	Type	Notes
id	BigAutoField	Primary key
reporter	FK -> User	CASCADE
target_type	CharField(16)	user/listing/message/review
target_id	BigAutoField	ID of reported object
reason	CharField(120)	Short reason
description	TextField(2000)	Detailed explanation
status	CharField(12)	pending/reviewing/resolved/rejected
admin_notes	TextField(2000)	Staff notes
reviewed_by	FK -> User	SET_NULL, nullable

7. Authentication System (JWT + OTP)

7.1 JWT Token Flow

The app uses JSON Web Tokens (JWT) via `django-rest-framework-simplejwt`. Tokens are stored in the browser's `localStorage` under `'skillswap.tokens'`.

LOGIN FLOW:

1. User sends email/username + password to `POST /api/v1/auth/login/`
2. Backend validates credentials (generic error on failure)
3. Backend returns `{ access, refresh, user }`
4. Frontend stores tokens in `localStorage` with expiry timestamp
5. Access token sent as `'Authorization: Bearer <token>'` header

TOKEN REFRESH:

1. Access tokens expire after 30 minutes
2. Axios interceptor detects 401 response
3. Interceptor sends refresh token to `POST /api/v1/auth/refresh/`
4. Gets new access + refresh tokens (rotation)
5. Retries original request with new access token
6. Old refresh token is blacklisted

LOGOUT:

1. Frontend sends refresh token to `POST /api/v1/auth/logout/`
2. Backend blacklists the refresh token
3. Frontend clears `localStorage`
4. User is redirected to home page

7.2 Email Verification (OTP)

VERIFICATION FLOW:

1. User registers -> account created, `email_verified=False`
2. User requests OTP: `POST /api/v1/auth/email/verification/request/`
3. Backend generates 6-digit code, hashes it, stores hash, emails code
4. User enters code: `POST /api/v1/auth/email/verification/verify/`
5. Backend verifies with constant-time check (Django's `check_password`)
6. On success: `email_verified=True`, user can use marketplace

SECURITY:

- Only hashed OTP codes stored (bcrypt-like via `make_password`)
- Max 5 verification attempts per code
- Code expires after 10 minutes (configurable)
- Only one active OTP per user per purpose
- New OTP supersedes previous unused ones

7.3 Password Reset (OTP)

Same OTP mechanism as email verification, but for password reset. Request endpoint returns 204 regardless of email existence (prevents enumeration).

7.4 Suspension Enforcement

When a user is suspended (`is_active=False`), the custom JWT authentication class (`UserJWTAuthentication`) rejects their tokens with 'This account has been suspended.' This works even for existing valid tokens.

7.5 Permission Classes

- `IsAuthenticated`: Standard DRF (must be logged in)
- `IsEmailVerified`: Must be logged in AND `email_verified=True`
- `IsProviderOrStaff`: Only the listing owner or staff can modify
- `IsStaff`: Must be logged in AND `is_staff=True`
- `AllowAny`: No authentication required

8. API Routes - Complete Reference

All routes are under `/api/v1/`. Interactive docs at `/api/v1/schema/swagger-ui/` when the server is running.

8.1 Core

Method	URL	Auth	Description
GET	<code>/api/v1/health/</code>	None	Health check

8.2 Authentication

Method	URL	Auth	Description
POST	<code>/auth/register/</code>	None	Register new user
POST	<code>/auth/login/</code>	None	Login (email or username)
POST	<code>/auth/refresh/</code>	None	Refresh JWT token
POST	<code>/auth/logout/</code>	Auth	Blacklist refresh token
GET	<code>/auth/me/</code>	Auth	Get current user
POST	<code>/auth/email/verification/request/</code>	Auth	Request OTP
POST	<code>/auth/email/verification/verify/</code>	Auth	Verify OTP
POST	<code>/auth/password-reset/request/</code>	None	Request reset OTP
POST	<code>/auth/password-reset/verify/</code>	None	Reset password with OTP

8.3 Profiles

Method	URL	Auth	Description
GET	<code>/profiles/me/</code>	Verified	Read own profile
POST	<code>/profiles/me/</code>	Verified	Create own profile
PATCH	<code>/profiles/me/</code>	Verified	Update own profile
GET	<code>/profiles/</code>	None	List public profiles
GET	<code>/profiles/<username>/</code>	None	Public profile detail
GET	<code>/skills/</code>	None	Browse all skills

8.4 Listings

Method	URL	Auth	Description
GET	<code>/listings/</code>	None	Search listings (many filters)
POST	<code>/listings/</code>	Verified	Create listing
GET	<code>/listings/mine/</code>	Verified	My listings
GET	<code>/listings/<slug>/</code>	Provider	Listing detail
PATCH	<code>/listings/<slug>/</code>	Provider	Update listing
DELETE	<code>/listings/<slug>/</code>	Provider	Archive listing
GET	<code>/listings/<slug>/applications/</code>	Provider	View applications
POST	<code>/listings/<slug>/applications/</code>	Verified	Apply to listing

GET	/applications/mine/	Verified	My applications
GET	/applications/<pk>/	Verified	Application detail
PATCH	/applications/<pk>/	Verified	Accept/reject/withdraw
GET	/categories/	None	List active categories

8.5 Orders

Method	URL	Auth	Description
GET	/orders/	Verified	List orders (filter by role)
POST	/orders/	Verified	Create order
GET	/orders/<pk>/	Verified	Order detail
PATCH	/orders/<pk>/	Verified	Cancel/start/complete
POST	/orders/<pk>/checkout/	Verified	Open payment session
POST	/orders/<pk>/mock-confirm/	Auth	Dev: mark as paid
POST	/stripe/webhook/	None	Stripe webhook

8.6 Messaging

Method	URL	Auth	Description
GET	/threads/mine/	Verified	My threads
GET	/threads/unread-count/	Verified	Unread count
GET	/threads/<pk>/	Verified	Thread + mark read
GET	/threads/<pk>/messages/	Verified	Messages in thread
POST	/threads/<pk>/messages/	Verified	Send message

8.7 Notifications

Method	URL	Auth	Description
GET	/notifications/mine/	Verified	List notifications
GET	/notifications/unread-count/	Verified	Unread count
POST	/notifications/read-all/	Verified	Mark all read
POST	/notifications/<pk>/read/	Verified	Mark one read

8.8 Reviews

Method	URL	Auth	Description
POST	/reviews/	Verified	Write review
GET	/profiles/<username>/reviews/	None	Public reviews

8.9 Moderation

Method	URL	Auth	Description
GET	/reports/	Verified	My reports or all (staff)
POST	/reports/create/	Verified	File a report
GET	/reports/<pk>/	Verified	Report detail
POST	/reports/<pk>/	Staff	Resolve/reject report
POST	/moderation/users/<pk>/	Staff	Suspend/reactivate user
GET	/moderation/listings/	Staff	Moderation queue
POST	/moderation/listings/<pk>/	Staff	Moderate listing

8.10 Rate Limits (Throttling)

API endpoints are rate-limited to prevent abuse:

Endpoint	Limit
auth_login	10 requests/minute
auth_register	5 requests/minute
auth_otp_request	3 requests/minute
auth_otp_verify	10 requests/minute
listing_create	10 requests/minute
application_create	20 requests/minute
order_create	20 requests/minute
order_checkout	10 requests/minute
message_send	60 requests/minute
review_create	10 requests/minute

9. Frontend Architecture

9.1 Provider Tree

The app wraps components in this order (in main.tsx):

```
StrictMode
  -> QueryClientProvider (React Query)
    -> BrowserRouter (React Router)
      -> AuthProvider (Auth Context)
        -> App (Route definitions)
```

9.2 Key Architecture Decisions

- No component library - all UI is hand-rolled with Tailwind CSS
- Server state via React Query (no Redux/Zustand)
- Auth state via React Context (useAuth hook)
- URL state via useSearchParams for shareable filters
- Forms via react-hook-form + zod validation
- Single Axios instance with automatic token refresh interceptor
- Custom classNames() utility instead of clsx

9.3 API Client (src/api/client.ts)

Central Axios instance. Base URL from VITE_API_URL env var (default http://localhost:8000/api/v1). Handles:

- Request interceptor: attaches Bearer token if not expired
- Response interceptor: auto-refreshes on 401, retries request
- Deduplication: single refresh promise for concurrent requests
- Auth expiry event: dispatches 'skillswap:auth-expired' window event

9.4 Service Modules (src/services/)

Each backend app has a matching frontend service module:

- auth.ts: register, login, logout, OTP verification, password reset
- profiles.ts: CRUD, search, skills list
- listings.ts: CRUD, applications, my listings
- orders.ts: list, detail, create, transition, checkout, mock-confirm
- messaging.ts: threads, messages, send, unread count
- notifications.ts: list, unread count, mark read
- reviews.ts: create, for profile
- reports.ts: create, list, detail, review + admin moderation

10. Pages & Routing (21 Pages)

All routes defined in src/App.tsx. Routes wrapped in Layout (Navbar + Footer).

Path	Page Component	Access
/	HomePage	Public
/login	LoginPage	Public (redirect if authed)
/register	RegisterPage	Public
/forgot-password	ForgotPasswordPage	Public
/reset-password	ResetPasswordPage	Public
/marketplace	MarketplacePage	Public
/listing/:slug	ListingDetailPage	Public
/profile/:username	PublicProfilePage	Public
/email-verification	EmailVerificationPage	Protected
/profile	ProfilePage	Protected
/profile/edit	ProfileEditPage	Protected
/listings/mine	MyListingsPage	Protected
/listings/new	ListingFormPage	Protected
/listings/:slug/edit	ListingFormPage	Protected
/listings/:slug/applications	ListingApplicationsPage	Protected
/orders	OrdersPage	Protected
/orders/:id	OrderDetailPage	Protected
/messages	MessagesPage	Protected
/messages/:threadId	ConversationPage	Protected
/notifications	NotificationsPage	Protected
/applications	MyApplicationsPage	Protected
/reports	ReportsPage	Protected
/admin	AdminDashboard	Admin (is_staff)
/admin/reports	ReportsPage (admin)	Admin
/admin/listings	AdminDashboard	Admin

Route Guards

- ProtectedRoute: Redirects to /login if not authenticated; to /email-verification if email not verified
- AdminRoute: Requires is_staff=True, otherwise redirects to /

Page Details

HomePage

Hero banner with CTA buttons. Shows up to 8 newest listings in a grid. Feature section explaining Learn/Earn/Connect.

LoginPage

Form: identifier (email or username) + password. Redirects to previous page on success. Links to register and forgot password.

RegisterPage

Form: first_name, last_name, username (min 3), email, password (min 8), confirm_password. After registration, redirects to /login.

MarketplacePage

Full filter sidebar: search, category, price range, remote delivery, sort. URL-persisted filters. Paginated results (12 per page).

ListingDetailPage

Shows listing info, provider card with ratings, reviews. Right sidebar: price, apply form (verified users), edit button (owner), report widget.

ListingFormPage

Handles both create (/listings/new) and edit (/listings/:slug/edit). Fields: title, description, category, skills (comma-separated), price, currency, delivery time, remote, location, cover image.

OrderDetailPage

Full order detail with status, dates, parties. Actions: Pay (buyer), Cancel (either), Start Work (provider), Mark Delivered (provider). Review box for completed orders.

ConversationPage

Chat interface with polling (5-second interval). Messages shown as bubbles. Auto-scrolls to bottom on new messages.

ProfilePage / ProfileEditPage

View: shows all profile data, skills, links. Edit: form for all fields including avatar upload and skill tags.

AdminDashboard

Staff-only. Shows pending listing count, status filter, and listing moderation actions (Approve/Reject/Suspend/Remove).

11. Components Reference

11.1 Layout Components

Navbar (src/components/layout/Navbar.tsx)

Desktop: horizontal nav with Marketplace, Orders, Messages, Notifications (with unread badge), 'Sell a skill' button.
Mobile: horizontal scrollable row. Shows username + logout when authenticated.

Layout (src/components/layout/Layout.tsx)

Renders Navbar + main container (max-w-7xl) + Outlet (child routes) + Footer.

11.2 Common Components (src/components/common/)

Button

Variants: primary (indigo), secondary (gray), outline, danger (red), ghost. Sizes: sm, md, lg. Loading state with spinner.

Field / Input / Textarea / Select

Form field wrapper with label, error message (role='alert'), hint text. Input/Textarea/Select accept 'invalid' prop for red border styling.

Avatar

Displays image or initials. Sizes: sm, md, lg, xl. Helper `mediaUrl()` prefixes relative paths with API base URL.

Pagination

Previous/Page X of Y/Next buttons. Returns null if `totalPages <= 1`.

Stars / RatingInput

Stars: displays rating as filled star SVGs + count. RatingInput: interactive 5-star radiogroup for entering ratings.

Feedback Components

Card (white bordered container), PageHeader (title + subtitle + action buttons), Alert (info/success/warning/error), Spinner, LoadingState, ErrorState (with retry), EmptyState, Badge (gray/green/amber/red/blue/purple), `statusColor()` helper.

11.3 Listing Components

ListingCard (src/components/listings/ListingCard.tsx)

Card with 4:3 cover image (fallback to category initial), category label, title, description preview, price, delivery time, provider name. Links to `/listing/{slug}`.

12. State Management

12.1 Server State (React Query)

All API data is managed by TanStack React Query. QueryClient created in src/lib/queryClient.ts with defaults: retry=1, refetchOnWindowFocus=false, staleTime=30s.

QUERY KEY PATTERNS:

['listings', 'featured']	- Home page listings
['listings', 'search', {filters}]	- Marketplace search
['listings', 'mine']	- My listings
['listing', slug]	- Single listing detail
['categories']	- Category list
['applications', slug]	- Applications for listing
['applications', 'mine']	- My applications
['orders', {role, page}]	- Orders list
['order', id]	- Single order
['threads']	- Message threads
['thread', id]	- Single thread
['messages', threadId]	- Messages (5s poll)
['notifications']	- Notifications list
['notifications', 'unread']	- Unread count (30s poll)
['me']	- Current user profile
['profile', username]	- Public profile
['reviews', 'profile', username]	- Profile reviews
['reports', {admin, status}]	- Reports list
['admin', 'listings', status]	- Admin listing queue

12.2 Auth State (React Context)

AuthProvider in src/hooks/useAuth.tsx provides:

- user: Current user object (or null)
- isAuthenticated: Boolean (user !== null)
- isLoading: Boolean (checking auth on mount)
- login(identifier, password): Stores tokens + sets user
- logout(): Clears tokens + nulls user
- refetchUser(): Re-fetches /auth/me/ to refresh user data

12.3 Local State

Component-level useState/useForm for ephemeral UI: form values, filter inputs, toggle states, feedback messages, error states.

12.4 URL State

MarketplacePage uses useSearchParams for filter state (q, category, minPrice, maxPrice, remote, sort, page) so filters are shareable/bookmarkable.

13. Business Logic & State Machines

13.1 Application Lifecycle

STATES: pending -> accepted/rejected/withdrawn

FLOW:

1. Buyer applies to listing -> status: pending
2. Provider accepts -> status: accepted (notifies buyer)
OR Provider rejects -> status: rejected (notifies buyer)
OR Buyer withdraws -> status: withdrawn (notifies provider)

RULES:

- Cannot apply to own listing
- Only one pending application per buyer per listing
- Cannot apply if already accepted for this listing

13.2 Order Lifecycle (State Machine)

STATES: pending_payment -> paid -> in_progress -> completed
cancelled (from pending_payment)
refunded (from paid or in_progress)

FLOW:

1. Buyer creates order from accepted application
-> status: pending_payment, creates Payment + Thread
2. Buyer pays (checkout) -> status: paid
(or: either party cancels -> status: cancelled)
3. Provider starts work -> status: in_progress
(or: admin refunds -> status: refunded)
4. Provider marks delivered -> status: completed
(or: admin refunds -> status: refunded)
5. After completion: buyer can leave a review

TRANSITIONS (service functions):

cancel_order(order, cancelled_by) -> requires pending_payment
mark_order_paid(order) -> requires pending_payment
start_order(order) -> requires paid
complete_order(order) -> requires in_progress
refund_order(order) -> requires paid or in_progress (admin only)

13.3 Listing Moderation States

STATES: draft -> pending -> published -> rejected/suspended/archived

Staff actions:

- approve: pending -> published
- reject: pending -> rejected
- suspend: published -> suspended
- remove: any -> archived

13.4 Notification Events

Every lifecycle transition emits notifications via the `notify()` service:

- Order: created, paid, started, completed, cancelled, refunded
- Application: accepted, rejected, withdrawn
- Message: new_message
- Review: new_review
- Report: filed, resolved
- Moderation: listing approved/rejected/suspended/removed

13.5 Cross-App Communication Pattern

Services only import service functions from other apps (never models directly). The `notify()` function is the central event bus. This keeps apps loosely coupled.

14. Payments (Stripe + Simulation)

14.1 Gateway Abstraction

The `orders/services.py` module has a `gateway()` factory that returns either `StripeGateway` (when `STRIPE_API_KEY` is set) or `SimulationGateway` (when empty).

14.2 SimulationGateway (Development)

```
MODE: simulation
1. create_checkout_session(order) returns mock session
   -> URL points to /orders/{id}/mock-confirm/ (dev-only endpoint)
2. Buyer clicks 'Pay now' -> frontend calls mock-confirm endpoint
3. Order status changes to 'paid'
```

No real money changes hands. Perfect for testing.

14.3 StripeGateway (Production)

```
MODE: stripe
1. create_checkout_session(order) creates real Stripe Checkout Session
   -> Returns Stripe-hosted URL for payment
2. Buyer redirected to Stripe, completes payment
3. Stripe sends webhook to POST /api/v1/stripe/webhook/
4. Webhook handler verifies Stripe-Signature header
5. Looks up order via metadata.order_id
6. Calls mark_order_paid(order)
```

REQUIRES:

- `STRIPE_API_KEY` (Stripe secret key)
- `STRIPE_WEBHOOK_SECRET` (webhook signing secret)
- `STRIPE_SUCCESS_URL` / `STRIPE_CANCEL_URL` (redirect URLs)

15. Messaging System

15.1 Thread Creation

A Thread is automatically created when an Order is created (via `create_thread_for_order`). It's a `get_or_create` (idempotent) - one thread per order. Connects buyer and provider of the order.

15.2 Message Flow

1. User opens conversation page (/messages/:threadId)
2. Messages loaded, thread marked as read (mark_thread_read)
3. Messages poll every 5 seconds (refetchInterval: 5_000)
4. User types message, clicks Send
5. POST /api/v1/threads/{id}/messages/ with body
6. Backend creates Message, notifies other participant
7. Frontend invalidates query cache, messages refresh

15.3 Read Status

mark_thread_read() bulk-updates all messages in a thread where the current user is the recipient (sender != current user) and is_read=False. Sets is_read=True and read_at timestamp.

16. Notifications System

16.1 Notification Model

Generic notifications with verb (e.g. 'new_order'), target_type/target_id (polymorphic reference), actor (who triggered it), and data (JSON payload). Notifications are created by the notify() service function.

16.2 Frontend Integration

- Navbar polls unread count every 30 seconds
- NotificationsPage lists all notifications with mark-as-read
- notificationHref() maps target_type to URL (order->/orders, listing->/listing, etc.)
- Unread notifications have highlighted background
- Clicking a notification marks it as read and navigates to target

17. Moderation & Reports

17.1 Reporting

Any verified user can report a user, listing, message, or review. Reports have a reason and description. Only one open report per reporter per target.

17.2 Staff Actions

- View all reports (filterable by status and target type)
- Resolve or reject reports with admin notes
- Suspend or reactivate users (sets is_active=False)
- Moderate listings: approve, reject, suspend, or remove
- Listing moderation queue with status filtering

18. How to Edit/Modify Everything

18.1 Adding a New Database Field

STEP 1: Edit the model
File: backend/apps/<app>/models.py
Add your field to the model class

STEP 2: Create migration
Command: python manage.py makemigrations
This creates a new file in apps/<app>/migrations/

STEP 3: Apply migration
Command: python manage.py migrate

STEP 4: Update serializer
File: backend/apps/<app>/serializers.py
Add the field to the relevant serializer class

STEP 5: Update frontend types
File: frontend/src/types/index.ts
Add the field to the TypeScript interface

18.2 Adding a New API Endpoint

STEP 1: Create the view
File: backend/apps/<app>/views.py
Add a new class-based or function-based view

STEP 2: Add URL pattern
File: backend/apps/<app>/urls.py
Add: `path('your-endpoint/', YourView.as_view(), name='name')`

STEP 3: Create/update serializer if needed
File: backend/apps/<app>/serializers.py

STEP 4: Add to frontend service
File: frontend/src/services/<app>.ts
Add a new exported function calling `client.get/post/etc.`

STEP 5: Use in a page/component
File: frontend/src/pages/<Page>.tsx
Use `useQuery/useMutation` with your new service function

18.3 Adding a New Frontend Page

STEP 1: Create the page component

File: frontend/src/pages/YourPage.tsx
Export default function YourPage() { ... }

STEP 2: Add route

File: frontend/src/App.tsx
Import YourPage and add:
<Route path="/your-path" element={<ProtectedRoute><YourPage /></ProtectedRoute>} />

STEP 3: Add navigation link

File: frontend/src/components/layout/Navbar.tsx
Add a <Link> in the desktop and/or mobile nav sections

18.4 Adding a New Django App

STEP 1: Create the app

Command: cd backend && python manage.py startapp your_app apps.your_app

STEP 2: Register in settings

File: backend/config/settings.py
Add 'apps.your_app' to INSTALLED_APPS

STEP 3: Create models, views, serializers, services, urls

Follow the pattern of existing apps:

- models.py for database tables
- services.py for business logic
- serializers.py for data validation
- views.py for API handlers
- urls.py for URL patterns

STEP 4: Include URLs in root router

File: backend/config/urls.py
Add: path('api/v1/', include('apps.your_app.urls'))

STEP 5: Create and run migrations

Command: python manage.py makemigrations your_app
Command: python manage.py migrate

18.5 Modifying the Auth System

TO ADD A NEW PERMISSION:

File: backend/apps/<app>/permissions.py (or users/permissions.py)

Create a class extending BasePermission:

```
class MyPermission(BasePermission):
    def has_permission(self, request, view):
        return request.user.some_condition
```

TO MODIFY JWT SETTINGS:

File: backend/config/settings.py

Edit the SIMPLE_JWT dict:

- ACCESS_TOKEN_LIFETIME: how long access tokens last
- REFRESH_TOKEN_LIFETIME: how long refresh tokens last
- ROTATE_REFRESH_TOKENS: issue new refresh on use
- BLACKLIST_AFTER_ROTATION: invalidate old refresh

TO ADD A NEW AUTH ENDPOINT:

1. Add view in apps/users/views.py
2. Add URL in apps/users/urls.py
3. Add serializer in apps/users/serializers.py
4. Add service function in apps/users/services.py

18.6 Modifying the Payment System

TO ADD A NEW PAYMENT PROVIDER:

File: backend/apps/orders/services.py

1. Create a new gateway class:

```
class PayPalGateway:
    mode = 'paypal'
    def create_checkout_session(order): ...
    def handle_webhook(request): ...
```

2. Update the gateway() factory function:

```
def gateway():
    if settings.PAYPAL_CLIENT_ID:
        return PayPalGateway()
    elif settings.STRIPE_API_KEY:
        return StripeGateway()
    return SimulationGateway()
```

3. Add env vars in settings.py and .env.example
4. Add webhook URL in config/urls.py

18.7 Modifying the Frontend Theme

FILE: frontend/src/index.css

The primary color palette is defined in @theme block:

```
@theme {
  --color-primary-50: #eef2ff;
  --color-primary-100: #e0e7ff;
  ...
  --color-primary-900: #312e81;
}
```

To change the primary color:

1. Go to <https://tailwindcss.com/docs/customizing-colors>
2. Pick a new color palette
3. Replace the --color-primary-* values
4. All components using primary-* classes will update automatically

18.8 Adding a New Notification Type

1. In the relevant service function, add a notify() call:

```
from apps.notifications.services import notify
notify(
    recipient=other_user,
    verb='your_verb',
    actor=request_user,
    target_type='model_name',
    target_id=obj.id,
    data={'key': 'value'},
)
```
2. In frontend NotificationsPage.tsx, add a case in notificationHref():

```
case 'your_verb': return `/target/${n.target_id}`
```
3. Optionally add the verb to the badge color mapping in statusColor()

18.9 Adding a New Filter to Marketplace

BACKEND:

1. File: backend/apps/listings/views.py -> ListingListCreateView
2. Add query param parsing in get_queryset()
3. Add .filter() call on the queryset

FRONTEND:

1. File: frontend/src/pages/MarketplacePage.tsx
2. Add to searchParams reading
3. Add filter UI element in the sidebar
4. Add to the query key and API call
5. Add to updateParams() for URL persistence

18.10 Common Editing Patterns

- Changing text/labels: Edit the component in frontend/src/pages/ or components/
- Changing API response shape: Edit serializer in backend/apps/<app>/serializers.py
- Adding validation: Add validators to serializer fields or model clean() method

- Changing page layout: Edit the JSX in the page component (Tailwind classes)
- Adding admin features: Create views with IsStaff permission, add URLs
- Changing order status flow: Edit service functions in orders/services.py

19. Deployment (Docker)

19.1 Docker Compose Services

docker-compose.yml defines 4 services:

1. db (PostgreSQL 16 Alpine)
 - Port: 5432
 - Volume: postgres_data (persistent)
 - Health check: pg_isready
2. backend (Django + gunicorn)
 - Port: 8000
 - Depends on: db (healthy)
 - Volumes: staticfiles, media
3. frontend (React build + nginx)
 - Port: 8080
 - Depends on: backend
 - Proxies /api/ and /admin/ to backend
 - Serves static/media from shared volumes
4. redis (Redis 7 Alpine) [optional]
 - Port: 6379
 - Profile: redis (docker compose --profile redis up)

19.2 Production Build

```
BACKEND DOCKERFILE:
- Base: python:3.13-slim
- Installs production requirements
- CMD: migrate -> collectstatic -> gunicorn (3 workers)

FRONTEND DOCKERFILE (multi-stage):
- Build stage: Node 22, npm ci, npm run build
- Runtime stage: nginx 1.27
- Copies dist/ to nginx html directory
- nginx.conf handles SPA routing + API proxying
```

19.3 CI Pipeline (.github/workflows/ci.yml)

Two parallel jobs:

- Backend: Python 3.13, PostgreSQL service, ruff lint/format, pytest, schema validation
- Frontend: Node 22, npm ci, oxlint, vitest, TypeScript check, Vite build

20. Testing

20.1 Backend Tests

```
# Run all tests
cd backend && pytest

# Run specific app tests
pytest apps/users/test_api.py

# Run with verbose output
pytest -v

# Run specific test
pytest apps/users/test_api.py::TestLogin::test_success
```

Tests use pytest-django with a test database (automatically created and destroyed).

20.2 Frontend Tests

```
# Run all tests
cd frontend && npx vitest run

# Run in watch mode
npx vitest

# Run specific test file
npx vitest run src/utils/format.test.ts
```

Tests use Vitest with jsdom environment and Testing Library for component tests.

21. Troubleshooting

Common Issues

Backend won't start

- Check PostgreSQL is running: `pg_isready`
- Check `.env` `DATABASE_*` variables match your PostgreSQL setup
- Run migrations: `python manage.py migrate`
- Check for port conflicts: something else on port 8000?

Frontend can't connect to backend

- Ensure backend is running on port 8000
- Check `VITE_API_URL` in frontend (`.env` or default)
- Check `CORS_ALLOWED_ORIGINS` includes `http://localhost:5173`
- Check browser console for CORS errors

Email OTP not received

- In development, emails print to console (terminal output)
- Check the terminal running the backend for OTP codes
- To use real email, configure `EMAIL_HOST`, `EMAIL_PORT`, etc. in `.env`

Payment simulation not working

- Ensure `STRIPE_API_KEY` is empty in `.env` (uses simulation mode)
- Use the 'Pay now' button -> 'Mock confirm' in development
- Check that `/orders/<id>/mock-confirm/` endpoint is accessible

Database migration errors

```
# If migrations conflict:
python manage.py makemigrations --merge

# If you need to reset (DESTRUCTIVE - deletes all data):
python manage.py migrate <app> zero
python manage.py migrate
```

Docker issues

```
# Full rebuild:
docker compose down -v # Removes volumes too
docker compose up --build

# View logs:
docker compose logs backend
docker compose logs frontend
docker compose logs db
```

Getting Help

- API Documentation: <http://localhost:8000/api/v1/schema/swagger-ui/>
- Django Admin: <http://localhost:8000/admin/>
- Architecture docs: docs/ARCHITECTURE.md
- CI status: Check GitHub Actions tab